

Design Document



Project Title: Gamified App For Student

Author: Shuai Jiang

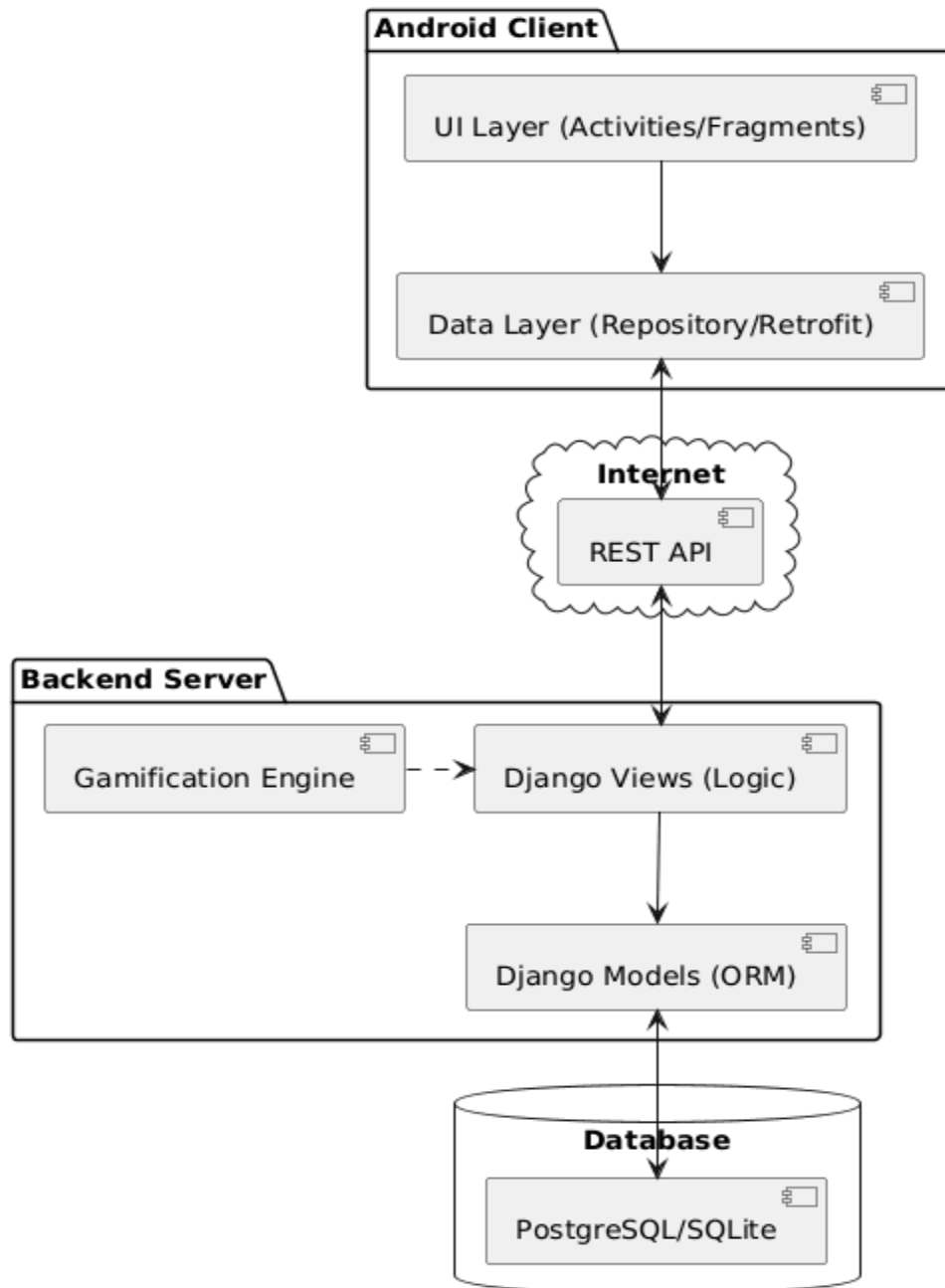
Date: 14/11/2025

1. System Architecture.....	3
2. Database Design.....	4
2.1 Entity-Relationship Diagram.....	4
2.2 Database Schema & SQL Scripts.....	5
3. Component Design (UML Class Diagram).....	7
3.1 Backend Class Diagram (Django Models).....	7
4. Sequence Diagrams.....	8
4.1 User Registration.....	8
4.2 User Login.....	9
4.3 User Logout.....	10
4.4 View Tasks.....	11
4.5 Create Task.....	12
4.6 Complete Task.....	13
4.7 Focus Session.....	14
4.8 View Leaderboard.....	15
4.9 Bluetooth Device Scanning.....	16
5. API Interface Design.....	17
6. SQL Statement.....	18
6.1 Account Management.....	18
6.2 Productivity Tools.....	19
6.3 Real-World Activities.....	21
6.4 Gamification Engine.....	22
6.5 SQL Operations & Tables.....	24

1. System Architecture

The system follows a **Decoupled Client-Server Architecture**:

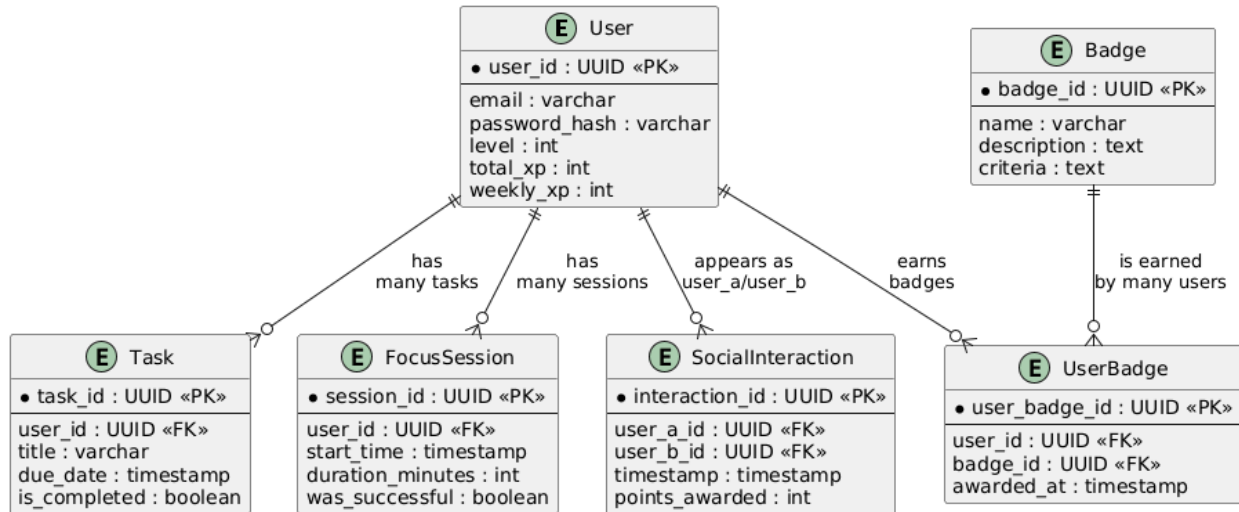
- **Client (Frontend)**: Native Android Application (Kotlin). Responsible for UI rendering and user input.
- **Server (Backend)**: Python Django Application. Responsible for business logic, gamification rules, and database management.
- **Database**: SQLite (Development) / PostgreSQL (Production). Stores all persistent data.
- **Communication**: RESTful API (JSON over HTTPS).



2. Database Design

2.1 Entity-Relationship Diagram

This diagram illustrates the logical relationships between data entities.



2.2 Database Schema & SQL Scripts

The following SQL statements represent the schema design for the core tables. Although Django ORM handles migration automatically, these scripts define the underlying structure.

(1) Users Table Stores user credentials and gamification stats.

```

CREATE TABLE users_user (
    id UUID PRIMARY KEY,
    email VARCHAR(255) UNIQUE NOT NULL,
    password_hash VARCHAR(255) NOT NULL,
    username VARCHAR(150),
    level INT DEFAULT 1,
    current_xp INT DEFAULT 0,
    total_credits INT DEFAULT 0,
    joined_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
  
```

(2) Tasks Table Stores the to-do list items linked to a user.

```

CREATE TABLE productivity_task (
    id UUID PRIMARY KEY,
    user_id UUID REFERENCES users_user(id) ON DELETE CASCADE,
    title VARCHAR(255) NOT NULL,
    description TEXT,
    due_date TIMESTAMP,
    is_completed BOOLEAN DEFAULT FALSE,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
  
```

(3) Focus Sessions Table Records study sessions for gamification calculation.

```
CREATE TABLE productivity_focussession (
  id UUID PRIMARY KEY,
  user_id UUID REFERENCES users_user(id) ON DELETE CASCADE,
  start_time TIMESTAMP NOT NULL,
  duration_minutes INT NOT NULL,
  status VARCHAR(20)
);
```

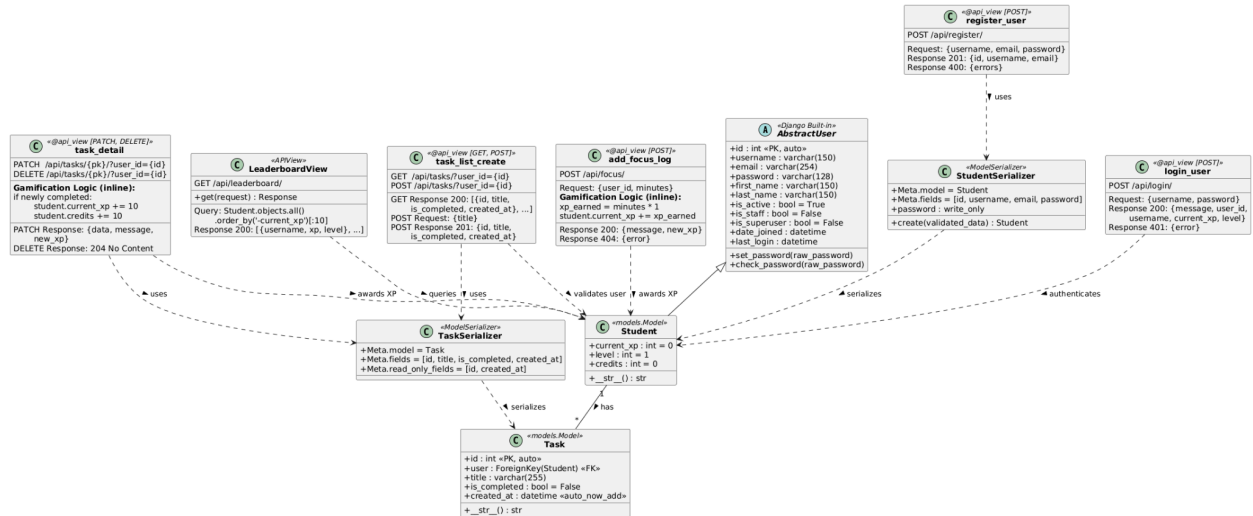
(4) Social Interactions Table Stores the Bluetooth-confirmed interactions.

```
CREATE TABLE social_interaction (
  id UUID PRIMARY KEY,
  initiator_id UUID REFERENCES users_user(id),
  receiver_id UUID REFERENCES users_user(id),
  confirmed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  points_awarded INT DEFAULT 0,
  CONSTRAINT unique_daily_interaction UNIQUE (initiator_id, receiver_id,
DATE(confirmed_at))
);
```

3. Component Design (UML Class Diagram)

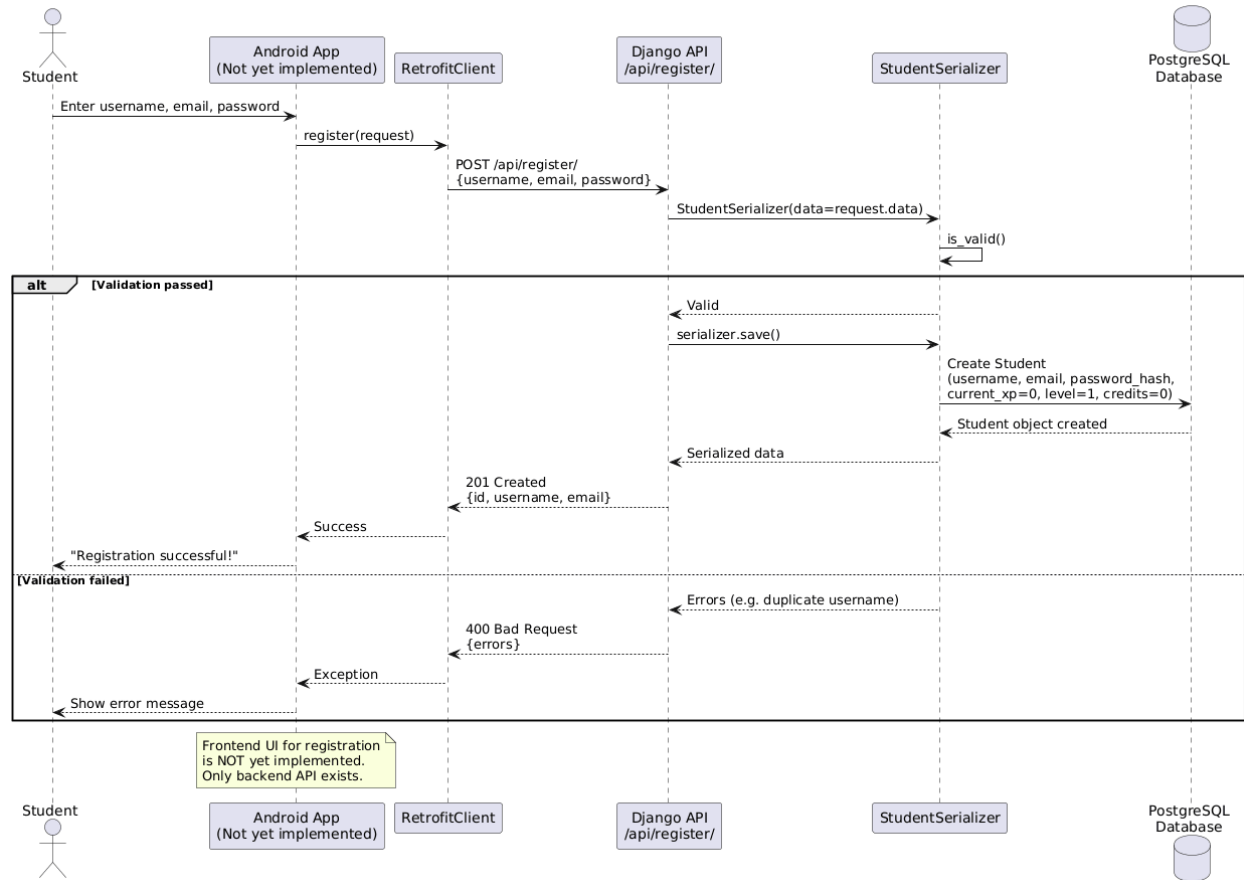
3.1 Backend Class Diagram (Django Models)

This diagram shows how the Python classes map to the database.

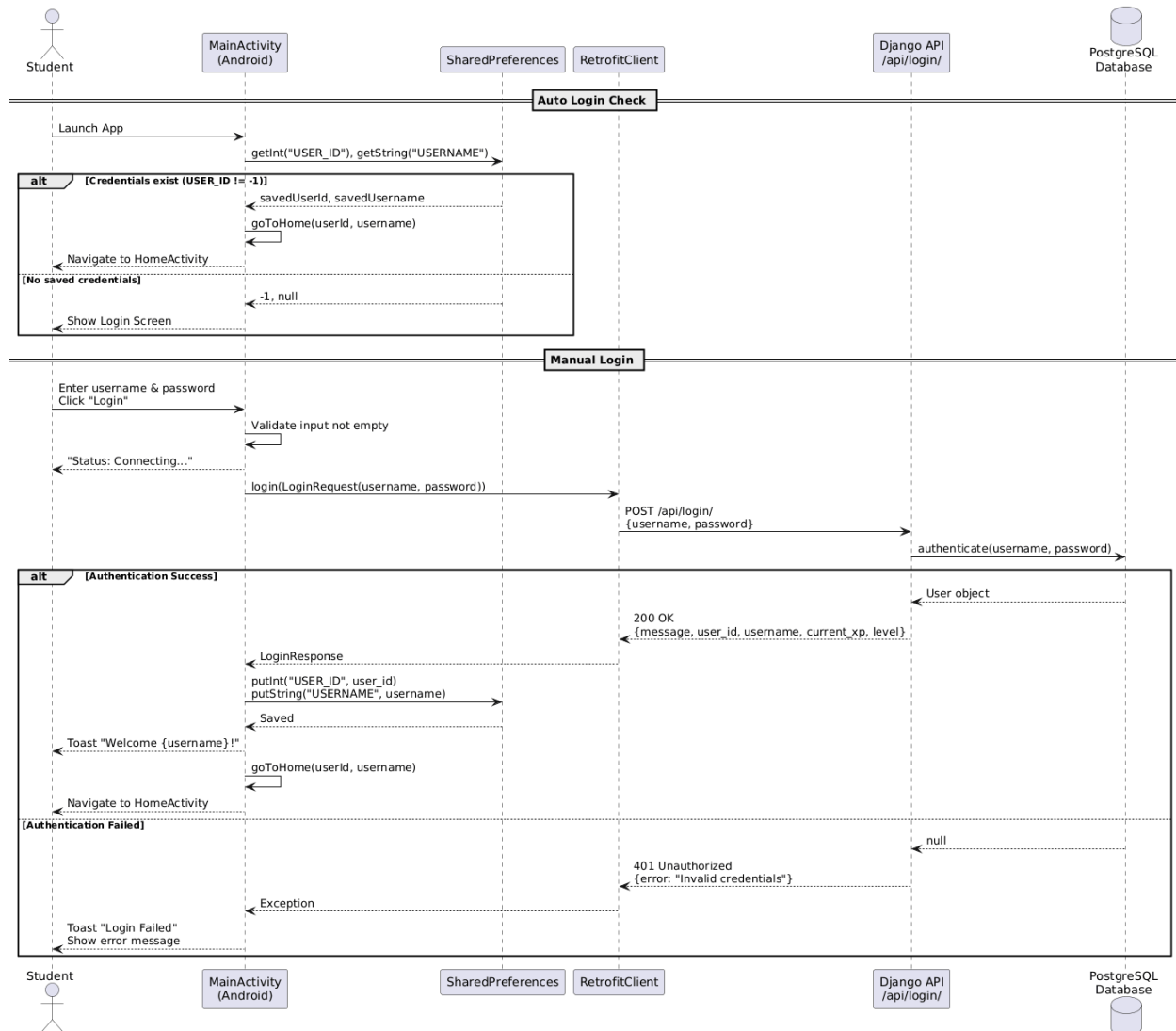


4. Sequence Diagrams

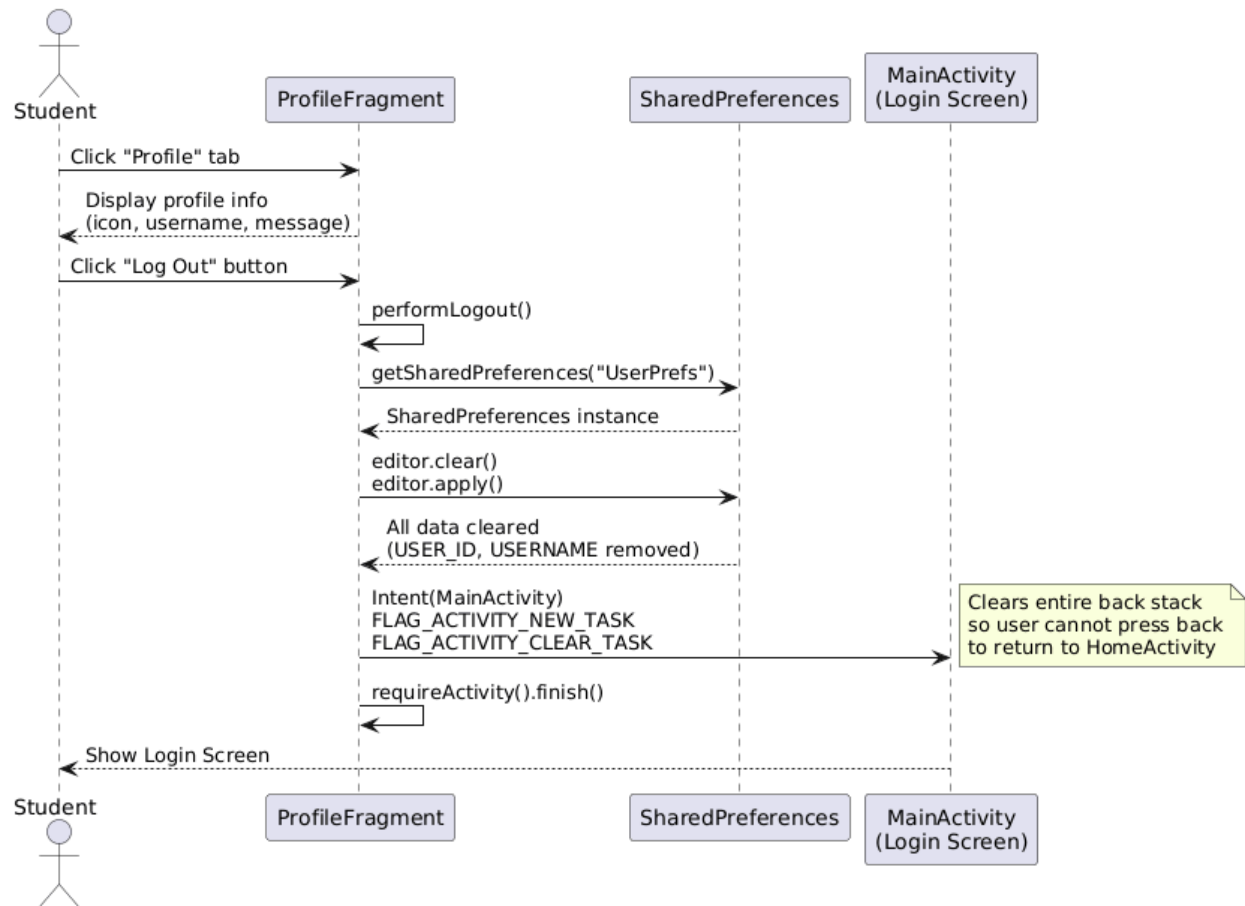
4.1 User Registration



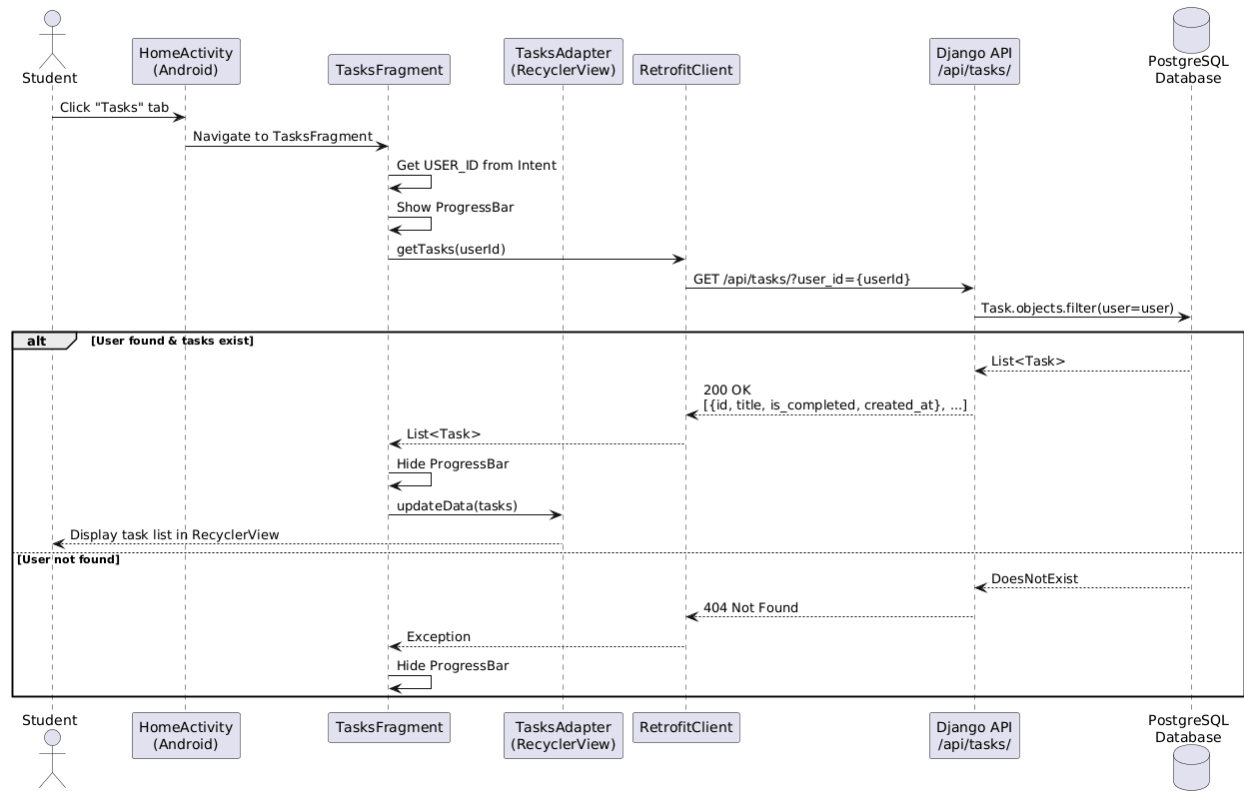
4.2 User Login



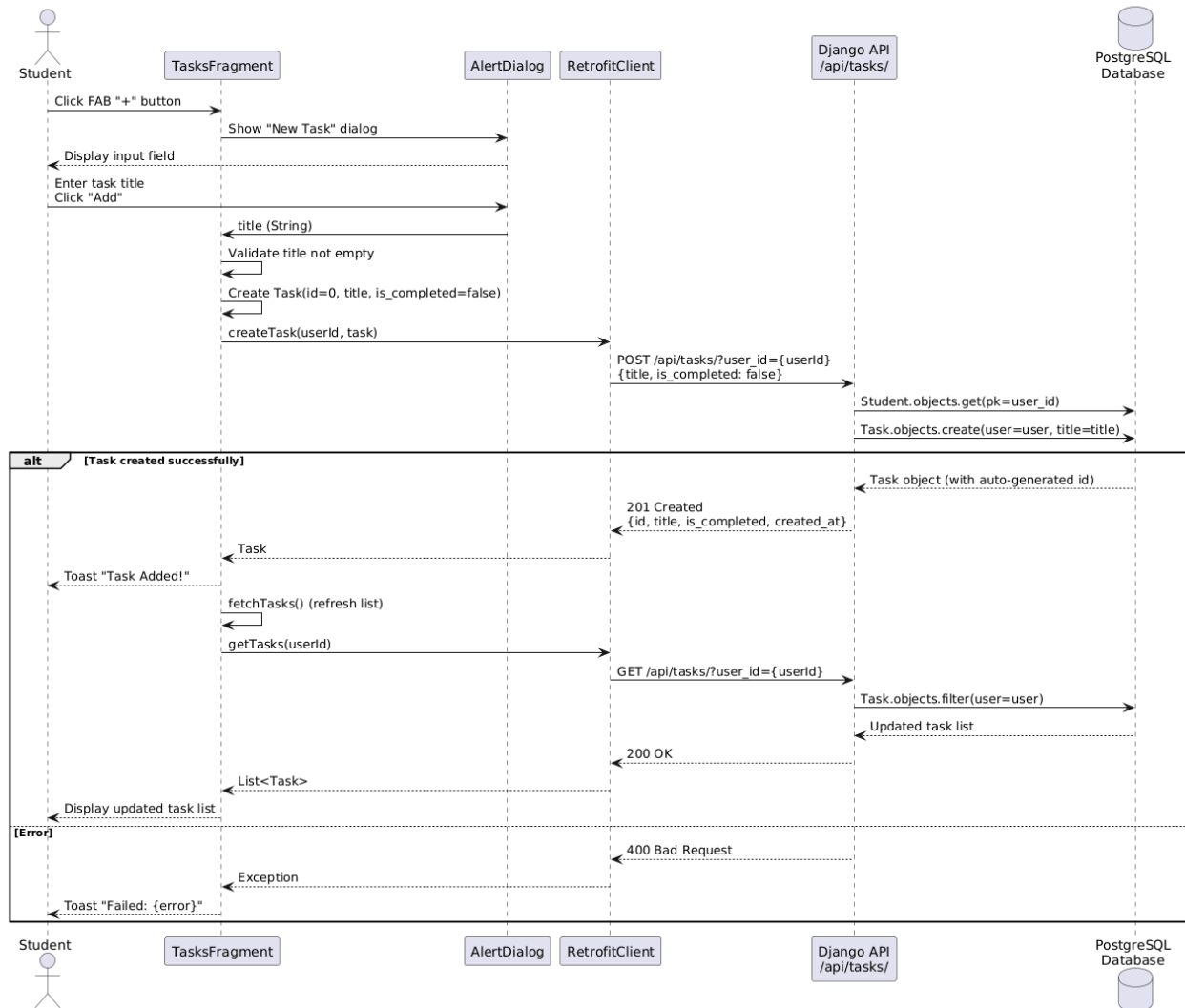
4.3 User Logout



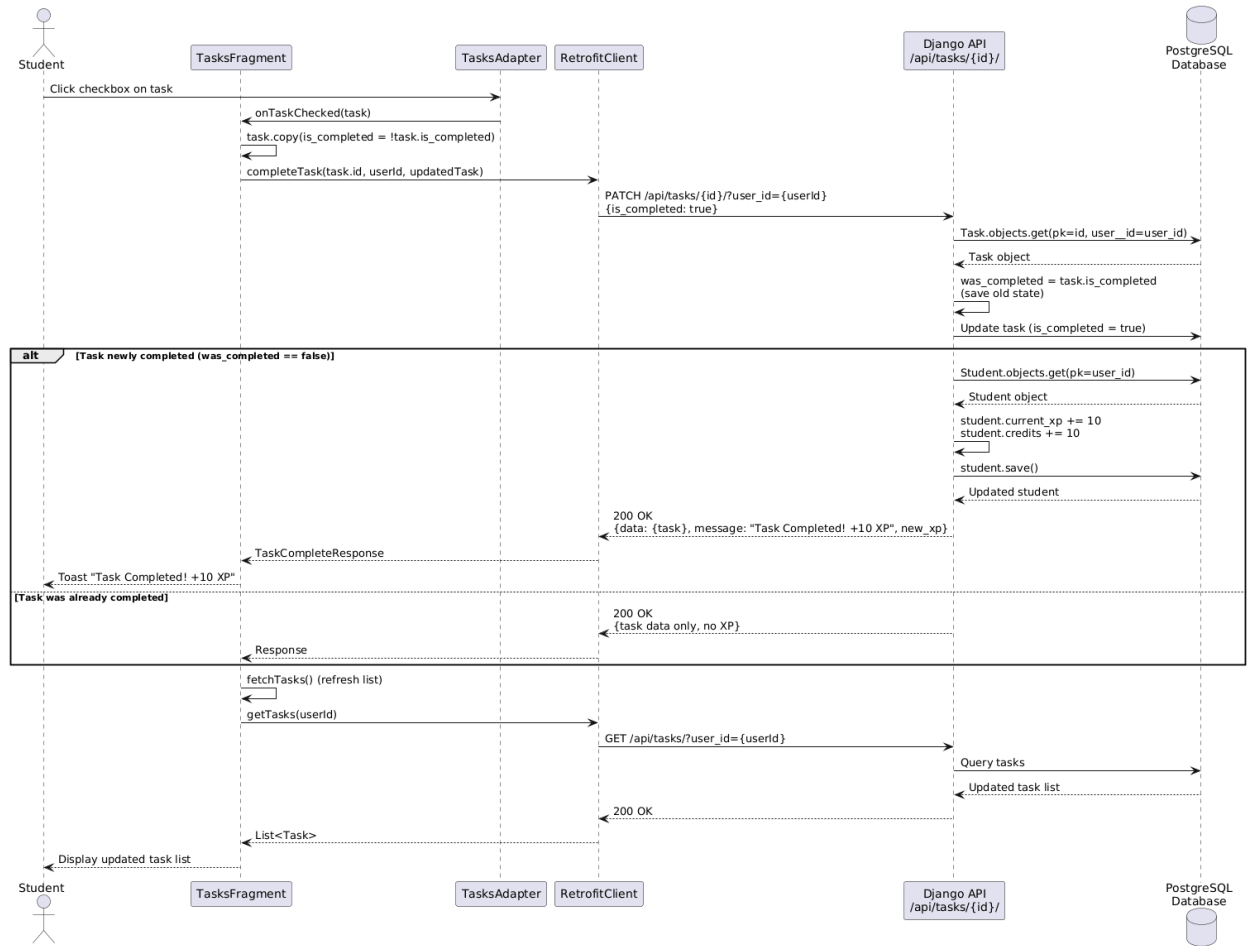
4.4 View Tasks



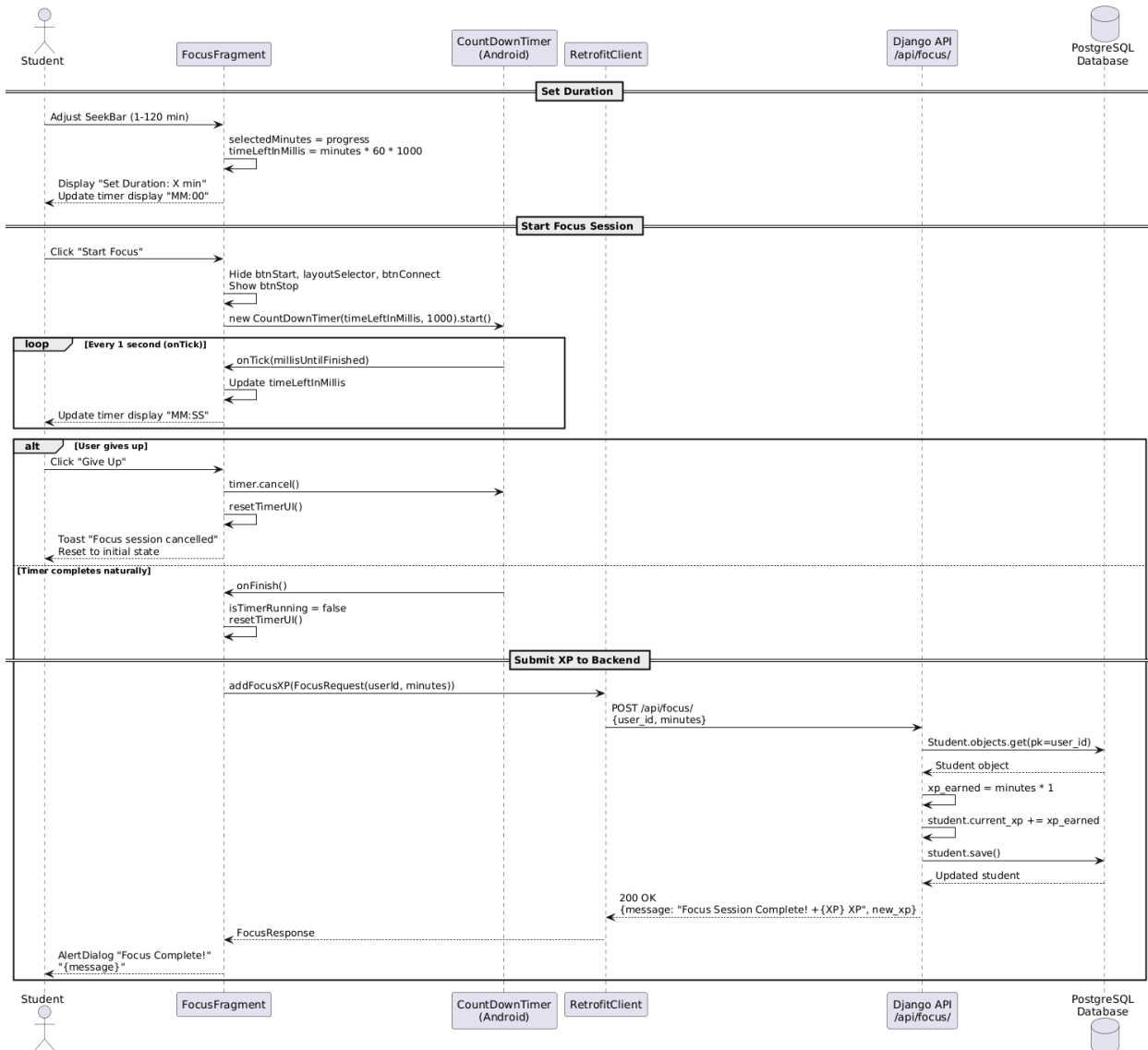
4.5 Create Task



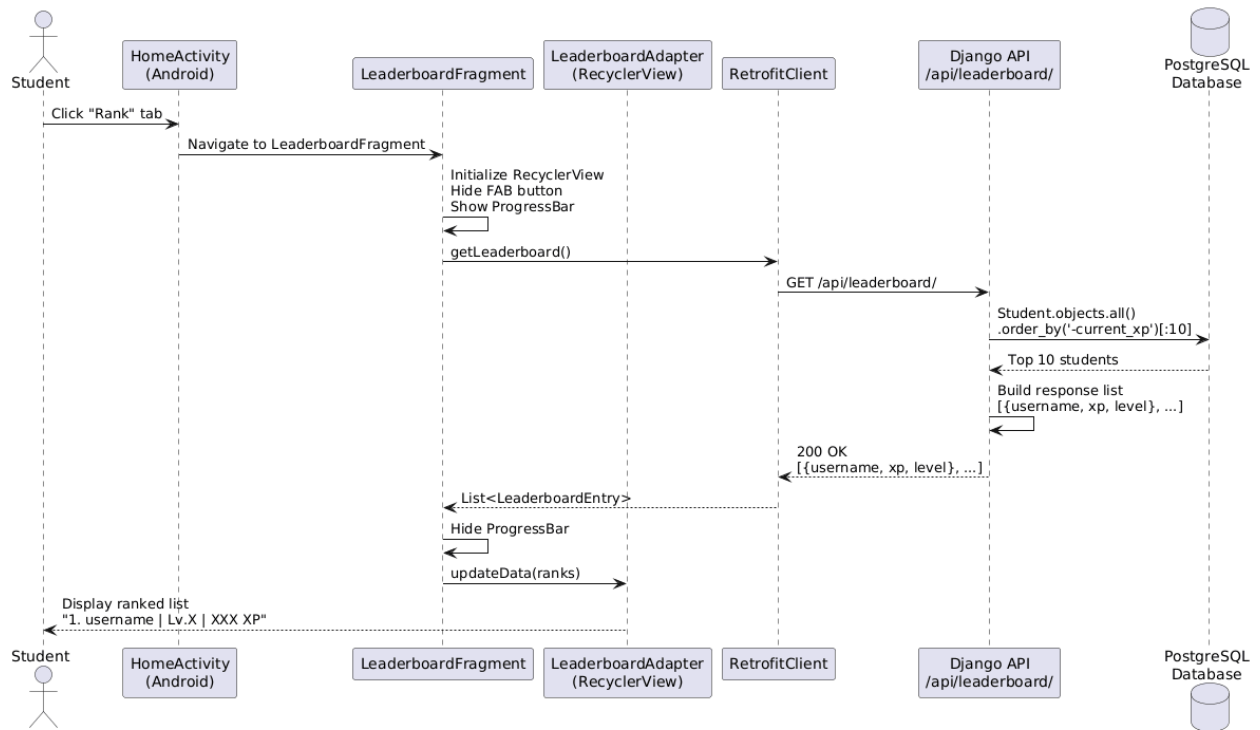
4.6 Complete Task



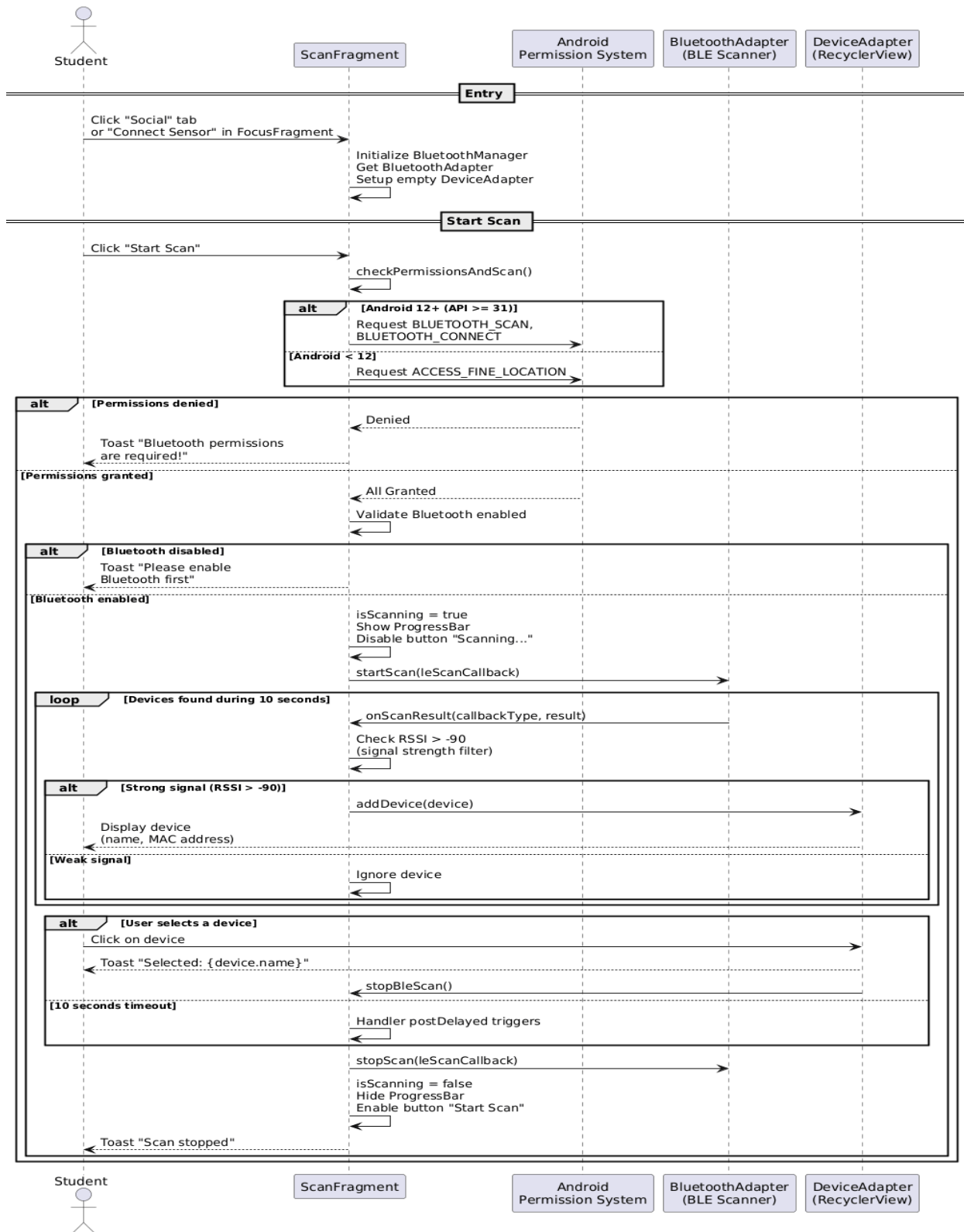
4.7 Focus Session



4.8 View Leaderboard



4.9 Bluetooth Device Scanning



5. API Interface Design

HTTP Method	Endpoint	Description	Request Body	Response
POST	/api/auth/register/	Create new user	{email, password}	{token, user_id}
POST	/api/auth/login/	Login user	{email, password}	{token}
GET	/api/tasks/	Get user's tasks	-	[{id, title, status}...]
POST	/api/tasks/	Create task	{title, due_date}	{id, title}
PATCH	/api/tasks/{id}/	Update task status	{is_completed: true}	{points_earned: 10}
POST	/api/social/confirm/	Log interaction	{target_user_id}	{status: "confirmed"}

6. SQL Statement

6.1 Account Management

UC-AM-01: Register for a new account

Insert a new user with hashed password and default gamification stats

```
INSERT INTO users_user (id, email, password_hash, username, level,
current_xp, total_credits, joined_at)
VALUES (:generated_uuid, :email, :hashed_password, :username, 1, 0, 0,
CURRENT_TIMESTAMP);
```

UC-AM-02: Log in with email and password

Step 1: Retrieve the user record by username for credential verification

```
SELECT id, username, email, password_hash, current_xp, level
FROM users_user
WHERE username = :username;
```

Step 2 (application-level): Django's authenticate() compares the provided password against the stored password_hash using PBKDF2. No additional SQL is issued.

UC-AM-03: Log out

No SQL statements required.

Logout is handled entirely on the client side by clearing SharedPreferences (user_id, username). No server-side session or token is invalidated in the current implementation.

UC-AM-04: View user profile

Retrieve the user's profile data including gamification stats

```
SELECT id, username, email, level, current_xp, total_credits, joined_at
FROM users_user
WHERE id = :user_id;
```

(Future) Retrieve the user's earned badges

```
SELECT b.id, b.name, b.description, ub.awarded_at
FROM user_badge ub
JOIN badge b ON ub.badge_id = b.id
WHERE ub.user_id = :user_id;
```

6.2 Productivity Tools

UC-PD-01: Add a new personal task

Step 1: Verify the user exists

```
SELECT id FROM users_user WHERE id = :user_id;
```

Step 2: Insert the new task linked to the user

```
INSERT INTO productivity_task (id, user_id, title, description, due_date,
is_completed, created_at)
VALUES (:generated_uuid, :user_id, :title, :description, :due_date,
FALSE, CURRENT_TIMESTAMP);
```

UC-PD-02: Mark a task as complete

This use case involves multiple SQL operations in sequence, including triggering UC-GM-01:

Step 1: Retrieve the task and verify ownership

```
SELECT id, user_id, title, is_completed, created_at
FROM productivity_task
WHERE id = :task_id AND user_id = :user_id;
```

Step 2: Mark the task as completed

```
UPDATE productivity_task
SET is_completed = TRUE
WHERE id = :task_id;
```

Step 3: Award XP and Credits to the user (triggers UC-GM-01)

```
UPDATE users_user
SET current_xp = current_xp + 10,
    total_credits = total_credits + 10
WHERE id = :user_id;
```

UC-PD-03: Add a new class to the weekly timetable

Status: Not yet implemented (Good to Have). Based on the design, the SQL would be:

(Planned) Insert a new class entry into the timetable

```
INSERT INTO timetable_class (id, user_id, class_name, location,
day_of_week, start_time, end_time)
VALUES (:generated_uuid, :user_id, :class_name, :location, :day_of_week,
:start_time, :end_time);
```

(Planned) Retrieve all classes for a user's weekly timetable

```
SELECT id, class_name, location, day_of_week, start_time, end_time
FROM timetable_class
WHERE user_id = :user_id
ORDER BY day_of_week, start_time;
```

UC-PD-04: Receive a notification before a class starts

No SQL statements required at notification time.

This use case relies on the data already stored by UC-PD-03. The client-side scheduler reads cached timetable data and triggers a local push notification. The relevant read query would be:

(Planned) Query upcoming classes for notification scheduling

```
SELECT class_name, location, day_of_week, start_time
FROM timetable_class
WHERE user_id = :user_id AND day_of_week = :today;
```

UC-PD-05: Start a focus session

Record the start of a new focus session

```
INSERT INTO productivity_focussession (id, user_id, start_time,
duration_minutes, status)
VALUES (:generated_uuid, :user_id, CURRENT_TIMESTAMP, :duration_minutes,
'in_progress');
```

UC-PD-06: Successfully complete a focus session

Step 1: Retrieve the user

```
SELECT id, current_xp FROM users_user WHERE id = :user_id;
```

Step 2: Update the focus session status to completed

```
UPDATE productivity_focussession
SET status = 'completed'
WHERE id = :session_id AND user_id = :user_id;
```

Step 3: Award XP based on duration (1 XP per minute)

```
UPDATE users_user
SET current_xp = current_xp + (:duration_minutes * 1)
WHERE id = :user_id;
```

UC-PD-07: Fail or interrupt a focus session

Step 1: Mark the focus session as failed

```
UPDATE productivity_focussession
SET status = 'failed'
WHERE id = :session_id AND user_id = :user_id;
```

Step 2: Apply credit penalty to the user (UC-GM-02)

```
UPDATE users_user
SET total_credits = GREATEST(total_credits - :penalty_amount, 0)
WHERE id = :user_id;
```

6.3 Real-World Activities

UC-RW-01: View daily step count

No SQL statements required.

Step count is read from the device's native health service (e.g., Google Fit API). If a step goal is reached, the gamification logic (UC-GM-01) is triggered:

(Planned) Award XP when daily step goal is reached

```
UPDATE users_user
SET current_xp = current_xp + :step_goal_bonus
WHERE id = :user_id;
```

UC-RW-02: Enable or disable the social interaction feature

No SQL statements required.

This is a client-side toggle stored in SharedPreferences. It controls whether the Bluetooth scanner is active. No server-side state change occurs.

UC-RW-03: Mutually confirm a social interaction (Bluetooth)

Backend code reference: Based on social_interaction schema in Design Document:

Step 1: Check if a duplicate interaction already exists today

```
SELECT id FROM social_interaction
WHERE initiator_id = :user_a_id
      AND receiver_id = :user_b_id
      AND DATE(confirmed_at) = CURRENT_DATE;
```

Step 2: Record the confirmed social interaction

```
INSERT INTO social_interaction (id, initiator_id, receiver_id,
confirmed_at, points_awarded)
VALUES (:generated_uuid, :user_a_id, :user_b_id, CURRENT_TIMESTAMP,
:points);
```

Step 3: Award XP to both participants

```
UPDATE users_user
SET current_xp = current_xp + :points
WHERE id = :user_a_id;

UPDATE users_user
SET current_xp = current_xp + :points
WHERE id = :user_b_id;
```

6.4 Gamification Engine

UC-GM-01: Receive points after completing a rewarded action

This is a cross-cutting concern triggered by UC-PD-02, UC-PD-06, and UC-RW-03. The core SQL pattern is:

Award XP and Credits to the user after a rewarded action

```
UPDATE users_user
SET current_xp = current_xp + :points_awarded,
    total_credits = total_credits + :points_awarded
WHERE id = :user_id;
```

UC-GM-02: Lose points (Credits) after failing a challenge

Status: Not yet implemented. Based on the design:

Deduct credits as a penalty (ensure non-negative)

```
UPDATE users_user
SET total_credits = GREATEST(total_credits - :penalty_amount, 0)
WHERE id = :user_id;
```

UC-GM-03: Receive a notification upon leveling up

Status: Level-up threshold logic not yet implemented. Based on the design:

Step 1: Check if the user's XP meets the next level threshold

```
SELECT id, current_xp, level
FROM users_user
WHERE id = :user_id;
```

Step 2: If current_xp >= threshold_for_next_level, update the level

```
UPDATE users_user
SET level = level + 1
WHERE id = :user_id AND current_xp >= :xp_threshold_for_next_level;
```

UC-GM-04: View the collection of all available badges

Status: Not yet implemented. Based on the ER diagram (Badge, UserBadge entities):

Step 1: Retrieve all available badges in the system

```
SELECT id, name, description, criteria
FROM badge;
```

Step 2: Retrieve badges earned by the current user

```
SELECT badge_id, awarded_at
FROM user_badge
WHERE user_id = :user_id;
```

The application joins these two result sets to display earned badges in color and unearned badges greyed out.

UC-GM-05: View the user's current rank on the Leaderboard

Retrieve the top 10 users ranked by XP (descending)

```
SELECT username, current_xp AS xp, level
FROM users_user
ORDER BY current_xp DESC
LIMIT 10;
```

UC-GM-06: Browse and purchase virtual items with in-app credits

Status: Not yet implemented (Discretionary). Based on the design:

Step 1: Retrieve all available shop items

```
SELECT id, name, description, price_credits
FROM shop_item;
```

Step 2: Check the user's current credit balance

```
SELECT total_credits
FROM users_user
WHERE id = :user_id;
```

Step 3: Process the purchase (deduct credits + add to inventory)

```
UPDATE users_user
SET total_credits = total_credits - :item_price
WHERE id = :user_id AND total_credits >= :item_price;

INSERT INTO user_inventory (id, user_id, item_id, purchased_at)
VALUES (:generated_uuid, :user_id, :item_id, CURRENT_TIMESTAMP);
```

6.5 SQL Operations & Tables

Use Case	SQL Operations	Tables Involved
UC-AM-01	INSERT	users_user

UC-AM-02	SELECT	users_user
UC-AM-03	(none - client-side)	--
UC-AM-04	SELECT	users_user, user_badge, badge
UC-PD-01	SELECT + INSERT	users_user, productivity_task
UC-PD-02	SELECT + UPDATE (x2)	productivity_task, users_user
UC-PD-03	INSERT	timetable_class (planned)
UC-PD-04	SELECT	timetable_class (planned)
UC-PD-05	INSERT	productivity_focussession
UC-PD-06	UPDATE (x2)	productivity_focussession, users_user
UC-PD-07	UPDATE (x2)	productivity_focussession, users_user
UC-RW-01	(none - device API)	--

UC-RW-02	(none - client-side)	--
UC-RW-03	SELECT + INSERT + UPDATE (x2)	social_interaction, users_user
UC-GM-01	UPDATE	users_user
UC-GM-02	UPDATE	users_user
UC-GM-03	SELECT + UPDATE	users_user
UC-GM-04	SELECT (x2)	badge, user_badge
UC-GM-05	SELECT (ORDER BY + LIMIT)	users_user
UC-GM-06	SELECT (x2) + UPDATE + INSERT	shop_item, users_user, user_inventory (planned)